Solution for Comparing Plants

For simplicity of describing the solutions, all indices of the array r will be taken modulo n.

Subtask 1

We are given the height comparisons of each pair of adjacent plants. We can conclude that plant x is taller than plant y if either of the following holds:

- $r[x] = r[x+1] = \ldots r[y-1] = 0$
- $r[y-1] = \ldots = r[n-2] = r[n-1] = 1$ and $r[0] = r[1] = \ldots = r[x-1] = 1$

The condition for returning -1 is similar.

If none of the conditions hold, return 0.

Implementing this naively requires $O(nq)$ time. We can speed this up by using a static prefix sum array.

Subtask 2

The overall idea is to generate a possible configuration of heights.

First observe that the tallest plant x must satisfy $r[x] = 0$. This suggests the following algorithm:

- Repeat the following for n times:
  1. Look for some index x such that $r[x] = 0$
  2. Decrement $r[x-k+1], r[x-k+2], \ldots, r[x-1], r[x]$ by 1

Consider the input $n = 3, k = 2, r = [0, 1, 0]$. The tallest plant must be 2. However, we also have $r[x] = 0$. We therefore need to replace rule (1) with the following:

- Look for some index x such that $r[x] = 0$ and $r[x-k+1], r[x-k+2], \ldots, r[x-k+1] \neq 0$. (*)

The index x satisfying the above condition is always unique (and therefore the final configuration of heights is also unique). The order in which these x are chosen will give rise to a possible configuration of heights, in which we can answer all the queries by simply comparing the heights. Complexity $O(n^2)$.

Subtask 3

Same as above; except that we need to use a segment tree with lazy propagation to speed up the algorithm. However, it can be tricky to implement (*).

To do this, we choose a value t such that $r[t] = 0$. If there are multiple of them, choose the smallest. We can do a range minimum query to check if any of $r[t-k+1], \ldots r[t-1]$ is zero. If none of them are zero, we choose $x = t$.

If not, we let x be the smallest index among $t-k+1, \ldots, t-1$ such that $r[x] = 0$.

Other than the potentially more tedious implementation of (*), the rest of the algorithm remains the same. Complexity $O(n \log n)$.

Subtask 4

The implementation of (*) becomes more problematic here. To illustrate, suppose that $k = 2$ and $r[0] = r[1] = \ldots = r[n-2] = 0, r[n-1] = 1$.

If we choose the value $t = n-2$, we realise that $r[n-3] = 0$. However, we run into further problems because $r[n-4]$ is also zero.

As such, we need to recursively search for the correct index. Complexity $O(n \log n)$.

```
extract(x):
  while there is a y in {x-k+1, x-k+2, ..., x-1} such that r[y] == 0:
    extract(y)
  end
  decrement r[x-k+1], r[x-k+2], ..., r[x] by 1
```

The ordering of the heights is then given by the order in which the procedure returns (rather than the order the of being called). The queries can still be answered by a direct comparison of the heights.

Subtask 5

We first generate a possible configuration of heights $h[0, 1, \ldots n-1]$ using the techniques from the previous subtasks.

Define $d(x, y) = \min(|x-y|, |x+n-y|, |y+n-x|)$ to be the distance between plants x and y. Generate a directed graph with $x \to y$ if and only if $h[x] > h[y]$ and $d(x, y) < k$.

Lemma:

- The above graph generated does not depend on the choice of $h[]$. In other words, any two height arrays $h_1[]$ and $h_2[]$ consistent with $r[]$ will generate the same directed graph.

Proof:

- Let G be any directed graph consistent with the input $r[]$ (that is, for each i, there are exactly

$r[i]$ values of j among $i+1, i+2, \ldots i+k-1$ such that $j \to i$).

- We first observe that any valid configuration $h[]$ corresponds to a run of the algorithm (*).
- It is therefore sufficient to show that any run of the algorithm (*) generates a topological sort of this graph G.
- The proof is similar to Kahn's algorithm. When plant i is removed by the algorithm (*), the conditions for removing i guarantee that i must have zero in-degree.
- Since we only remove nodes with zero in-degree, the order generated must correspond to a valid topological sort of G.

To check if plant x is taller than plant y, we check if there exists a (directed) path from x to y.

To do so, we can compute the transitive closure using Flyod-Warshall and pre-compute the answers to the entire space of queries. Complexity $O(n^3)$.

Alternative solution to subtask 5

We follow the idea of (*). We look for some z such that $r[z] = 0$ and $r[z-k+1], r[z-k+1], \ldots, r[z-k+1] \neq 0$, but now including an additional constraint that $z \neq x$. We stop when x is the only plant we can remove.

There exists a configuration of heights such that plant y is taller than plant x if and only if the above algorithm removes plant x.

Take note that doing so naively would require $O(qn^2) = O(n^4)$ computation time, if we do not use a segment tree.

However, it is easy to speed this up to $O(n^3)$. For each plant $0 \leq x \leq n-1$, we pre-compute in advance all y such that y can possibly be taller than x, storing them in a $n \times n$ table.

Subtask 6

Same idea as subtask 5. However, we are unable to generate the entire graph as there can be up to $O(nk)$ edges.

For each plant x, define the 'left edge' and 'right edge' as follows:

- $left(x) =$ the tallest plant shorter than x among $x-k+1, \ldots, x-1$, or -1 if it does not exist
- $right(x) =$ the tallest plant shorter than x among $x+1, \ldots, x-k+1$, or -1 if it does not exist

We now generate a graph with edges $x \to left(x)$ and $x \to right(x)$ for each x. To check if plant x is taller than plant y, we check if there exists a (directed) path from x to y.

Since we only need to compare against plant 0, we can use single-source BFS or DFS to find all

vertices reachable from 0.

Complexity $O(n \log n)$ (the queries are answered in $O(1)$ time, however we need $O(n \log n)$ to generate the array $h[]$, each value of $left(x)$ and $right(x)$ requires $O(\log n)$ to compute.

We may also use the idea to the alternative solution of subtask 5. After finding all the plants that can be taller than plant 0, we can invert the ranks (i.e. replace $r[i]$ with $k - 1 - r[i]$ for each i) to check if a plant can shorter than plant 0.

Subtask 7

For each query (x, y), let z be the first value among $left(x), left(left(x)), left(left(left(x))), \ldots$ such that y is between z and $left(z)$.

If $h[z] > h[y]$, we can then conclude that x is taller than y.

We do so similarly for the right edges.

To speed up this computation, we use power-of-2 tables:

$$left_1(x) = left(x), left_{k+1}(x) = left_k(left_k(x))$$

Using this approach, we may compute z using $O(\log n)$ time.

Total complexity: $O(n \log n + q \log n)$.

To the readers who are keen for an extra challenge: figure out how to solve this problem using $O(n \log n)$ preprocessing time and $O(1)$ time per query!

Solution for Connecting Supertrees

Subtask 1

Construct any tree on n nodes.

Subtask 2

We know that $p[i][j] = 0$ if and only if i and j belong to different connected components. If $p[i][j] > 0$ and $p[j][k] > 0$, then we must have $p[i][k] > 0$; we may return impossible if this condition is not satisfied.

Once this condition is satisfied, we proceed to identify the connected components and construct any tree for each connected component.

Subtask 3

We proceed to identify the connected components as per the previous section. Instead of constructing a tree through each connected component, we construct a cycle instead.

Take note that connected components cannot have size 2; there are no cycles of length 2.

Subtask 4

As with the previous subtasks, we deal with each connected component separately.

For each i, consider the set of all j such that $p[i][j] = 1$. For each of these sets, we construct any tree through these nodes. For each of these trees, assign a special node which we will call the `root'.

For each connected component, we now construct a cycle through these roots.

Subtask 5

We need to check for an additional condition: if $p[i][j] = 1$ and $p[j][k] = 1$, then we must have $p[i][k] = 1$.

The previous checks from subtask 3 (consistency of connected components, no cycles of length 2) still apply as usual.

Subtask 6

If $p[i][j] = 3$ for some pair (i, j), then there must be another pair (i', j') such that $p[i'][j'] \geq 4$. Therefore, this cannot occur.

We simply return 0 whenever this occurs.

Solution for Carnival Tickets

Observe that the special card pulled by the game master is always equal to the median of the input.

Since n is even, we may assume that the value of each prize is given by the sum of the top half values, minus the sum of the bottom values.

We can now change the rules as follows(*):

- On each round, 1 ticket of each color is chosen.
- These n tickets are split into two piles labeled '+' and '-'.
- Each pile consists of exactly $n/2$ tickets
- The value of the prize is given by the sum of all tickets labeled '+', minus the sum of all tickets labeled '-'.

It is easy to see that a player who wishes to maximize the value of his prize will choose to put the tickets of larger value into the `+' pile. By doing so, the value of the prize remains the same as compared to the original version.

It is in fact convenient to relax the rules even further and combine the k rounds into a single round(**):

- These kn tickets are split into two piles labeled '+' and '-', each pile consisting of $kn/2$ tickets
- The value of the prize is given by the sum of all tickets labeled '+', minus the sum of all tickets labeled '-'.

It is clear that the maximal attainable value using the rules in (**) is at least as large the maximal value attainable in (*): this is because we can simply merge the '+' and '-' piles.

We claim that they are in fact equal (this would solve subtask 4).

To show that they are equal, we need a method for `splitting' up these tickets back into k piles. Starting with the $kn/2$ tickets, we proceed as follows until all the tickets are split into k rounds:

- Sort the colors by the total number of '+' in each pile.
- For each of the $n/2$ colors with the most total number of '+', take 1 ticket with a '+' label from each of them. For the remaining $n/2$ numbers, take 1 ticket with a '-' label.
- Collect the tickets from the previous step, and form 1 round with it.

It remains to solve the problem using the rules in (**).

We proceed using a greedy approach. Initially, the k smallest tickets of each color is assigned '-'. A step consisting of removing one of these '-' and replacing it with a '+' from the same color. We

proceed to greedily maximize the sum of all '+' minus the sum of all '-' at each step.

To execute this greedy algorithm, we record down the largest ticket in the '-' pile of that color and the largest unused ticket of that color. The increase in score is given by the sum of these two numbers, therefore we can choose the color which corresponds to the largest value.

If we do so naively, the running time is $O(n^2m)$. We can speed this up by using a priority queue.

Solution for Packing Biscuits

We say that a value of y is good if it is possible to pack x bags, each of y total tastiness.

Subtask 1

We proceed by checking every value of y, noting that $y \leq 10^5$. For each value of y, we use the `greedy coin change' algorithm to decide if it is good or not.

Implementing this naively gives a runtime of $\approx 10^5 \cdot x$ operations. There are two ways to speed it up.

One possible way is to remove x coins at a time instead of removing them one by one.

The other way is to check for values of y at most $10^5/x$, so that we have less values of y to check in the even that x is large. However, contestants will have to return 1 immediately if x is larger than 10^5 to avoid actually packing x empty bags!

Subtask 2

We begin with the following observation:

- If $a[i] \geq 3$, the solution remains unchanged if we decrease $a[i]$ by 2 and increase $a[i+1]$ by 1.

Using this observation, we may combine smaller biscuits into bigger ones such that $a[i] \in \{0, 1, 2\}$.

A second observation is the following:

- Suppose $a[i] = 0$ for some i. Let y be any integer, we perform division with remainder and write $y = 2^i \cdot q + r$. Then y is good if and only if $2^i \cdot q$ is good and r is good.
- Suppose $a[i] \neq 0$ for all i. Then for any integer y, y is good if and only if the total tastiness is at least y.

Using this observation, we may split the input into consecutive segments of non-zero values. The final answer is the product of the answers individual segments.

Subtask 3

Similar to the previous subtask, if $a[i] \geq x + 2$, we may decrement $a[i]$ by 2 and increase $a[i+1]$

by 1.

We now proceed by dynamic programming. Let $f(n,i)$ be the answer if we replace $a[0], a[1], \ldots, a[i-1]$ by 0 and increment $a[i]$ by n.

We obtain the following recurrence relation:

$$f(0,60) = 1, f(n,i) = \begin{cases} f(\lfloor \frac{n+a[i]}{2} \rfloor, i+1) & n + a[i] < x \\ f(\lfloor \frac{n+a[i]}{2} \rfloor, i+1) + f(\lfloor \frac{n+a[i]-x}{2} \rfloor, i+1) & n + a[i] \geq x \end{cases}$$

To explain the relation, let S be the set of all valid y when $a[0], a[1], \ldots a[i-1]$ are replaced by 0 and $a[i]$ incremented by n.

If y is a multiple of 2^{i+1}, then the biscuits of tastiness 2^i must be used in pairs. We therefore merge pairs of biscuits to make biscuits of tastiness 2^{i+1}.

If y is not a multiple of 2^{i+1} but not a multiple of 2^i, then we need to use x biscuits of tastiness 2^i. The remaining $a[i] + n - x$ biscuits must be used up in pairs.

If y is not a multiple of 2^i, then y cannot be good.

Subtask 4

Define

$$s[i] = \sum_{j=0}^{i} a[i] \cdot 2^i$$

to be the total tastiness the first $i+1$ types of biscuits.

We need another observation:

- Let $2^i \leq y < 2^{i+1}$. Then y is good if and only if $s[i] \geq x \cdot y$ and $y - 2^i$ is good.

The forward direction is clear. We will justify the backward direction. For simplicity, assume $a[i+1] = a[i+2] = \ldots = a[k-1] = 0$. Since $y - 2^i$ is good, consider some way to pack them, remove these biscuits from our collection.

We need to pack the remaining biscuits into x packs of 2^i each. We do so by merging smaller biscuits into bigger ones- whenever there are two biscuits of tastiness 2^j for some $j < i$, we replace them with a single biscuit of tastiness 2^{j+1}. It can be shown that we will end up with at least x biscuits of tastiness 2^i each after the merging.

We now have an efficient way to enumerate all the solutions. Let A_i be the set of good values which are at most 2^i. We then have

$$A_{i+1} = A_i \cup \{y | y - 2^i \in A_i \text{ and } s[i]/x \geq y\}$$

We can now explicitly list out all elements of A_i. Thus the time taken for a single query is linear in the size of the answer returned.

Subtask 5

Let $g(n)$ be the number of possible y which are less than n. The following recurrence relation solves the problem:

- Let $2^i < n \leq 2^{i+1}$. Then

$$g(n) = g(2^i) + g(\min(n, 1 + s[i]/x) - 2^i)$$

  with the initial values $g(n) = 0$ for all $n \leq 0$ and $g(1) = 1$.

By using a hash table to store all previously computed values, this algorithm runs in $O(k^2)$ time.

The time complexity is justified by the fact that once $g(2^0), g(2^1), g(2^2), \ldots g(2^{i-1})$ are computed, we only need $O(k)$ time to compute $g(2^i)$.

Solution for Counting Mushrooms

10 points

For each $1 \leq i \leq n-1$, query `use_machine([0,i])`. We can thus determine the species of every mushroom.

25 points

Same idea as above, except that we query `use_machine([i, 0, i+1])` for all odd values of i. As we only query odd values, the number of queries required is halved.

Beyond 25 points

The general strategy is as follows:

- Collect a set of k mushrooms which are known to be of the same species, let these mushrooms be $x_1, x_2, \ldots x_k$.
- We can now make a query of the form $[y_1, x_1, y_2, x_2, \ldots, y_k, x_k]$. The return value to this query reveals the number of type A mushrooms among $y_1, y_2, \ldots y_k$.

A possible strategy is to query $[0, i]$ for each $1 \leq i \leq 2 \cdot k - 2$, thus determining the species of the first $2k-1$ mushrooms. We can then find k mushrooms of the same type. The total number of queries is given by $2k - 2 + \lceil (n - 2k - 1)/k \rceil$. Optimizing over possible values of k gives 397 queries, achieved when $94 \leq k \leq 107$.

It is possible to further reduce the number of queries to $k + \lceil (n - 2k - 1)/k \rceil$. We first query $[0, 1]$ and $[0, 2]$. This gives us two types of mushrooms of the same species, call it x, y.

We can proceed by querying $[3, x, 4, y]$. The return value of this function will identify the species of both mushroom 3 and 4.

Using this approach, we can now find the species of the first $2k-2$ mushrooms with only k queries. By setting k to any value between 137 and 146, we use at most 281 queries.

Finally, observe that when we query $[y_1, x_1, y_2, x_2, \ldots, y_k, x_k]$, we gain information about the species of y_1 by looking at the parity of the return value.

Thus, we can repeatedly expand our set of known mushrooms and increase the value of k appropriately, using at most 245 queries when $73 \leq k \leq 94$.

100 points

Full solution identifies about 5 mushrooms in 2 queries during the first phase. There are constructive appoaches, but we can also try to brute-force all possible strategies that can achieve this. Suppose that we need to identify mushrooms A, B, C, D, E, and we have already identified x and y mushrooms of types 0 and 1 respectively. We can try all possible strings of characters $A, B, C, D, E, 0 (x \text{ times}), 1 (y \text{ times})$ to ask as a first query q_1. Depending on the answer to q_1, the second query q_2 should be able to unambiguously determine each type of A, B, C, D, E, so we can brute-force it as well (note that q_2 may depend on the result of q_1). The number of possible queries is small enough so that all decision trees can be processed within reasonable time.

To avoid overhead for identifying the first few mushrooms, we can actually have two decision trees:

- The first tree uses the only additional mushroom of type 0, and will either figure out that A, B, C, D, E are all type 0 in 1 query, or identify them in 3 queries.
- The second tree uses, when available, 3 mushrooms of type 0 and 1 mushroom of type 1 (or vice versa). It always identifies A, B, C, D, E in 5 queries.

We will employ the second tree whenever there are enough extra mushrooms, and the first one otherwise (that only happens when all identified mushrooms are type 0). This approach can unconditionally identify $5k$ mushrooms in $2k + 1$ queries. Combining this with ideas above and choosing k optimally, we can solve the problem in 226 queries.

Note that there is a much simpler randomized solution that also identifies ~ 2.5 mushrooms per query, but it can't be used because of adaptive grading (unless you can somehow confuse the grader...).

Beyond 100 points

Since the end of the contest a much better solution has been discovered by the community: https://codeforces.com/blog/entry/82924

Solution for Stations

Let us refer to the station with label L given by Ali as node L. The SIB network forms a tree. Suppose we root the tree at the station with the smallest label. We observe that if Brenda can, given 2 nodes A and B, determine whether

- node A is an ancestor of node B
- node B is an ancestor of node A
- nodes A and B are not ancestors of each other

then Brenda can solve each query as follows:

1. For the source node S, check each neighbouring node to see if it is an ancestor of node S.
    - If it is, it must be the parent of node S, which we shall call node P. There can be at most 1 such node.
    - Otherwise, it is a child of node S. Check if it is an ancestor of the destination node T. If yes, return this child.
2. If no answer is returned by the steps above, either node T is an ancestor of node S or they are not ancestors of each other. In either case, the answer is node P.

Hence, if Ali can label the stations such that Brenda can check the above just from the labels of the station, then Brenda can solve all the queries.

Subtask 1

The network is a line. Ali can find a leaf station and label the stations 0 to $N-1$ in DFS order in $O(N)$ time. Then, node A is an ancestor of node B if and only if $A \leq B$, which can be checked in $O(1)$ time.

Subtask 2

The network is a binary tree. For each station X in the input, Ali can give the label $X+1$. That way, the parent of node L is node $\lfloor L/2 \rfloor$. Hence, node A is an ancestor of node B if and only if $A \leq B$ and repeatedly applying $B \to \lfloor B/2 \rfloor$ reaches A before reaching 0. This takes $O(\log N)$ time per check.

Subtask 3

Ali can give label 0 to the station with degree more than 2, or to any station if no such station exists, and label its neighbours 1 to M where M is the degree of node 0. If we remove node 0, the

remaining stations will form disjoint lines where one station in each line is adjacent to node 0. For each line, let the station adjacent to node 0 be node L. Ali can label each station X in the line with $(d(L, X) \times 1000 + L)$ where $d(L, X)$ is the number of edges in the unique path between station X and node L. After this, node A is an ancestor of node B if and only if $A \leq B$ and either $A == 0$ or $(A\%1000 == B\%1000)$ which takes $O(1)$ time to check.

Subtask 4

The network is small with at most 8 stations. Select a random station and label it 1, this will be the root of the tree. For each node L with M children, label it's children $(L \times 8)$ to $(L \times 8 + M - 1)$. That way, the parent of node L is node $\lfloor L/8 \rfloor$. Hence, node A is an ancestor of node B if and only if $A \leq B$ and repeatedly applying $B \to \lfloor B/8 \rfloor$ reaches A before reaching 0. The maximum possible label is $8^7 \leq 10^9$.

Subtask 5

In the general case, select a random station to root the tree and perform a DFS from the root:

```
counter = 0
dfs(station v):
    in[v] = counter++
    call dfs(c) for each child c of v
    out[v] = counter++
call dfs(root)
```

Suppose Brenda knows $in[v]$ and $out[v]$ for the source, destination and the neighbours of the source. For any 2 stations v_1 and v_2, she can deduce that v_1 is an ancestor of v_2 if and only if $in[v_1] < in[v_2] < out[v_1]$, and thus be able to solve the query using the method described in the previous section. One simple way to encode this is that Ali can label each station v with $L = in[v] \times 1000 + out[v]$, so Brenda can get $in[v] = L/1000$ and $out[v] = L\%1000$. The maximum label will be $999 \times 1000 + 999 = 999999$ which will get around 65 points if the first 4 subtasks are handled separately.

To increase the points further, we can make a few observations (*)

- Brenda can find the parent of the source station s by looking for the neighbouring station v with the smallest $in[v]$ or $out[v]$, unless $in[s] == 0$ in which case station s is the root.
- If we sort the m children $c[]$ of station s in ascending order of $in[v]$ to get $c_1, c_2, \ldots, c_m$, then $out[c_1] = in[c_2] - 1, out[c_2] = in[c_3] - 1$ and so on. Also, $in[s] = in[c_1] - 1$ and $out[c_m] = out[s] - 1$.

Hence, some minor optimisations can be made to get $70 - 80$ points by not encoding some values of $out[v]$ and deriving them from the $in[v]$ of the other neighbours instead.

In order to get 89points, we need Ali to use a maximum label of 2000 or $2n$. We label each station v

as follows:

- If the number of edges in the unique path from station v to the root is even, label the station with $in[v]$.
- Otherwise, label the station with $out[v]$

For Brenda, there are 2 possible cases to consider:

- If the label of the source station s is smaller than the label of its neighbours, then the label of station s is $in[s]$ and we can use the observations in (*) to find the parent of station s (if station s is not the root) and $in[v]$ and $out[v]$ for all its children v and be able to solve the problem as described above.
- Otherwise, the label of station s is $out[s]$ and we can use the same idea to also find the parent of station s (if station s is not the root) and $in[v]$ and $out[v]$ for all its children and do the same.

To get the full 100 points, we make a final observation that the exact values of the labels are not essential since the 89 point solution only needs to know how each label compares with each other. Since the labels are distinct, we can simply label the stations 0 to $N-1$ based on the rank of the labels in the 89 point solution.